DK_BICYCLE_NETWORK · SCRIPTS/SQL/10A–10C

How every edge gets its LTS score

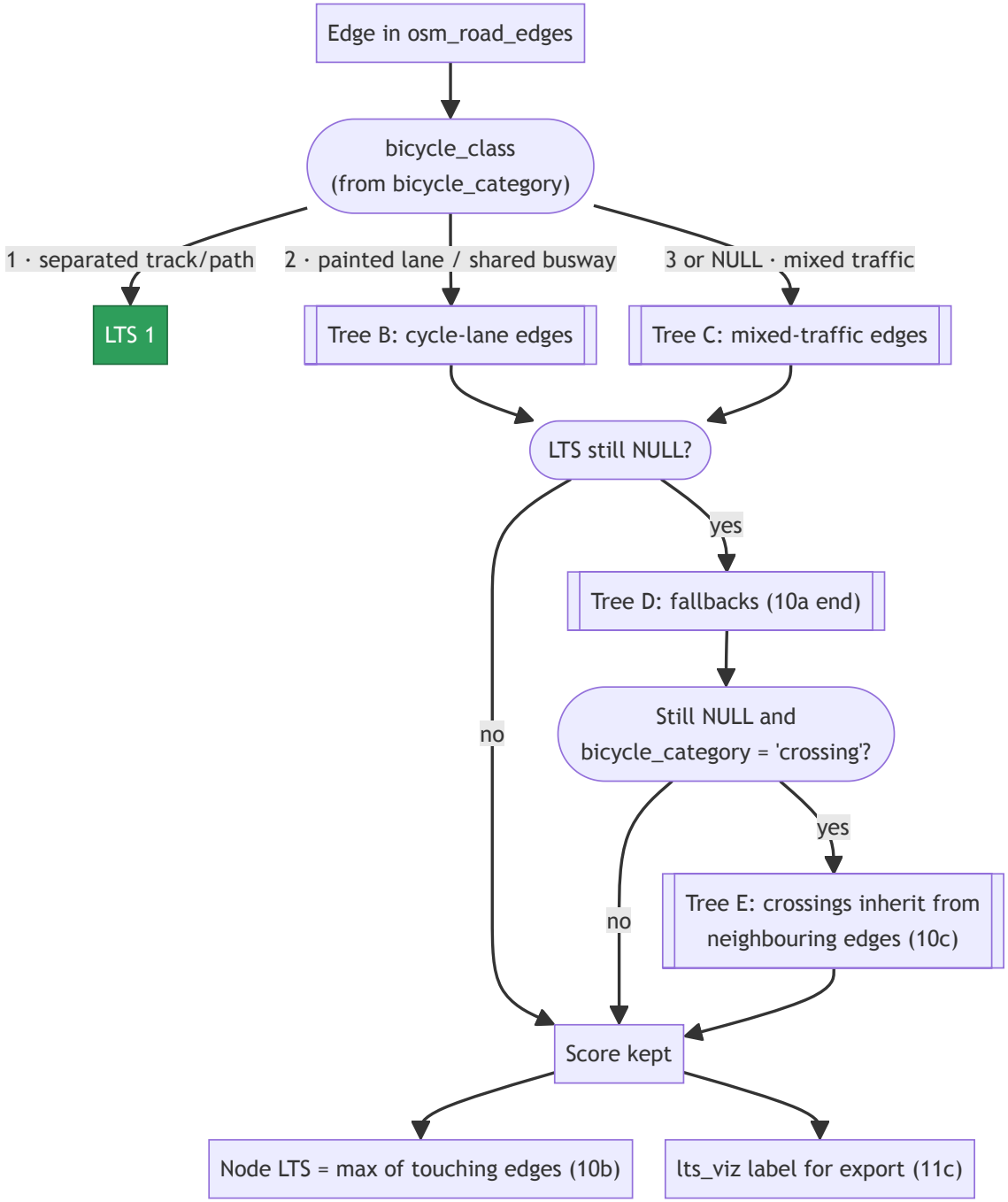
The Level of Traffic Stress (LTS) for each edge in `osm_road_edges` comes from a chain of SQL `UPDATE` statements. Later statements overwrite earlier ones, so the code reads as four separate rule lists. The trees below show what that chain actually does, with all overwrites already resolved.

LTS 1 · LTS 2 · LTS 3 · LTS 4 · 0 · bike-allowed path, no infra · 999 · no cars, no bikes

OVERVIEW

The full pipeline

Every edge passes through these stages in order. An edge leaves the pipeline at the first stage that gives it a score. Variable names match the SQL columns. *speed* means `maxspeed_assumed` and *lanes* means `lanes_assumed`.


Edge in osm_road_edges
bicycle_class
(from bicycle_category)
1 · separated track/path
2 · painted lane / shared busway
3 or NULL · mixed traffic
LTS 1
Tree B: cycle-lane edges
Tree C: mixed-traffic edges
LTS still NULL?
yes
no
Tree D: fallbacks (10a end)
Still NULL and
bicycle_category = 'crossing'?
yes
no
Tree E: crossings inherit from
neighbouring edges (10c)
Score kept
Node LTS = max of touching edges (10b)
lts_viz label for export (11c)

STEP 0 · 10A

Bicycle class

The class is set from `bicycle_category`. The script then asserts that every edge with `cycling_allowed = TRUE` has a class.

BICYCLE_CATEGORY	CLASS
`shared_track`, `cycleway`, `cycleway_shared`, `cycletrack`	1
`cyclelane`, `shared_busway`	2
`cycle_living_street`, `crossing`, `shared_lane`	3
NULL and `cycling_allowed = TRUE`	3
anything else, e.g. motorways and roads where cycling is banned	NULL

TREE A · CLASS 1

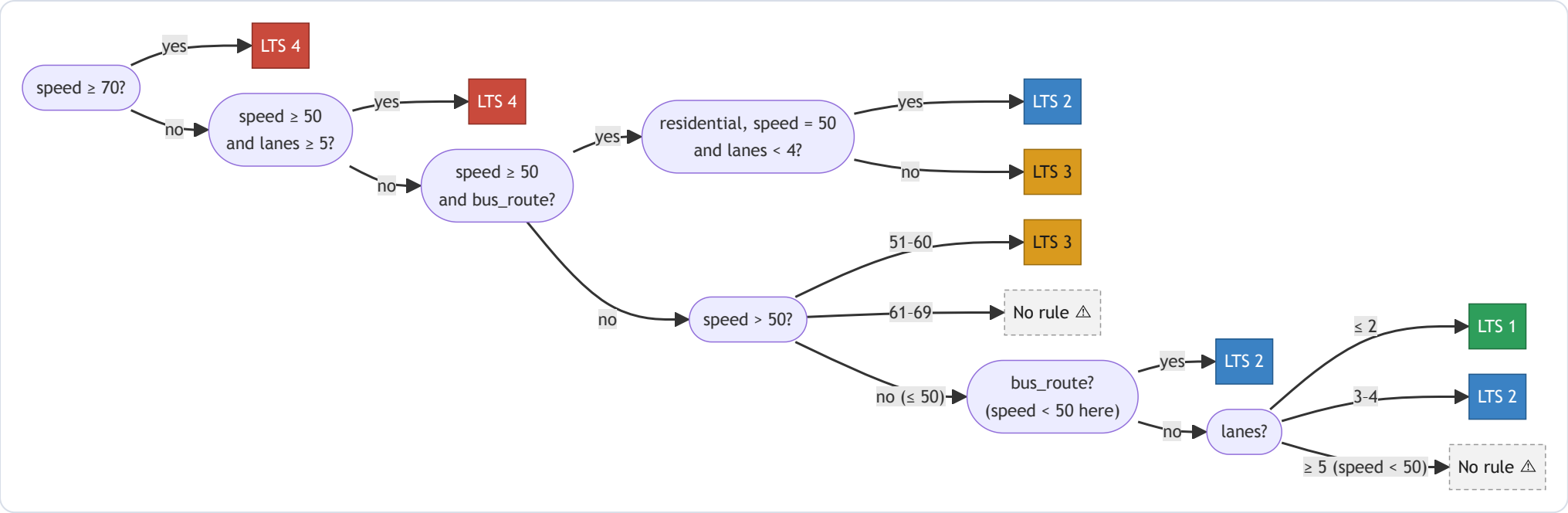
Separated infrastructure is always LTS 1

Any edge with `bicycle_class = 1` gets LTS 1, whatever its speed, lanes or bus route. None of the later rules can overwrite it, because they all require class 2, class 3/NULL, or a current LTS of 3 or 4.

TREE B · CLASS 2

Painted cycle lanes and shared busways

Read the tree top to bottom and take the first branch that matches. The branches are ordered so that the highest LTS that the SQL would assign comes first.



⚠ “No rule” means the edge goes on to the fallbacks in Tree D. Cycle lanes run on car roads, so in practice such an edge would trip the 10a assertion *“Car edges missing LTS value”*. Those speed and lane combinations must therefore not occur in the data.

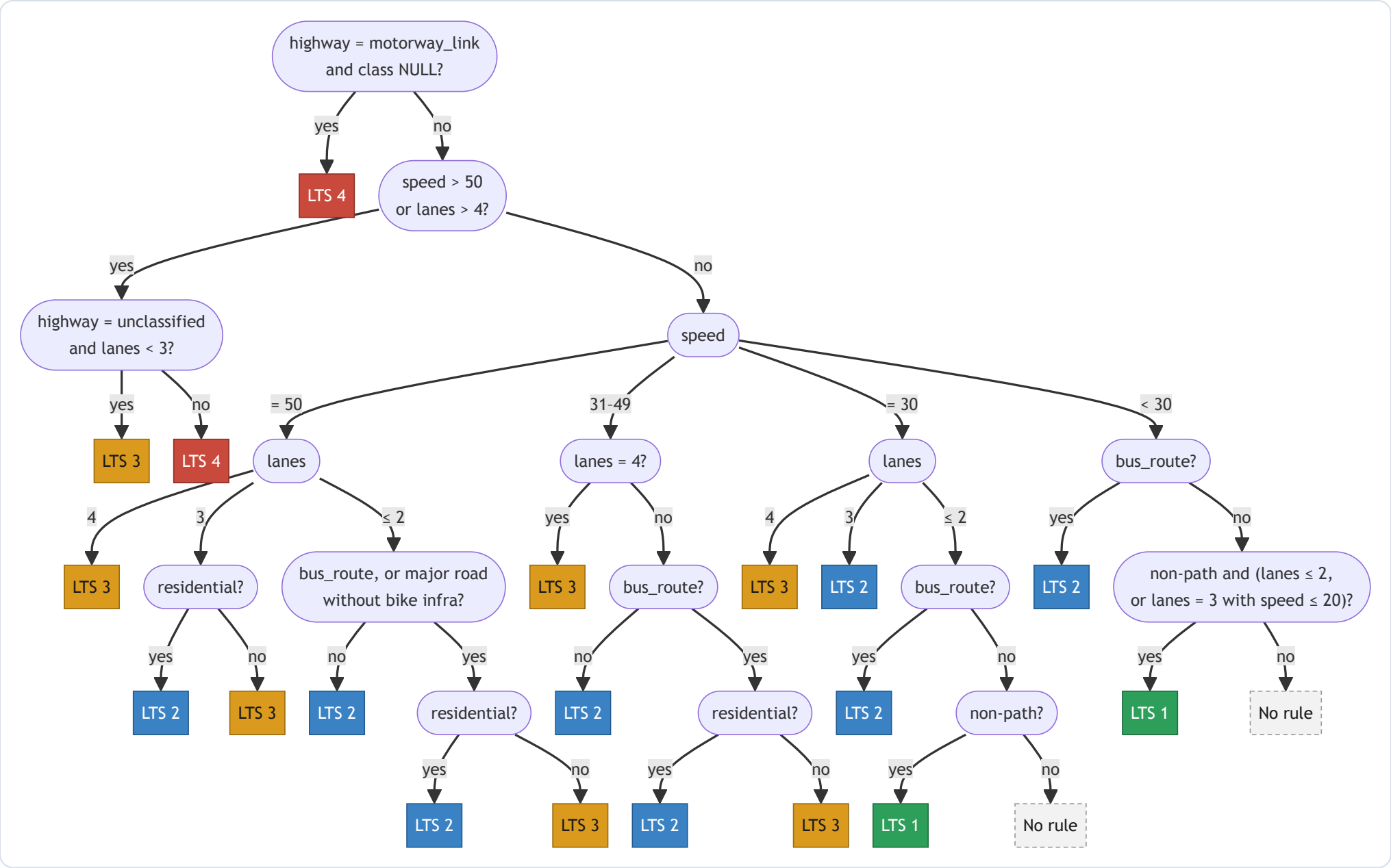
TREE C · CLASS 3 OR NULL

Mixed traffic: shared lanes, crossings, roads without cycle infrastructure

This tree also covers roads where cycling is not allowed, with `bicycle_class` NULL. The SQL comment says it directly: *LTS 1 does not consider whether cycling is allowed*. Those edges get LTS 1–4 here, and the export later labels them `no_cycling`.

“Non-path” below means `bicycle_infrastructure_final = TRUE` **or** `highway` is not one of `path`, `bridleway`, `footway`, `pedestrian`, `steps`. *“Major road”* means `primary`, `secondary`

or `tertiary`, or their `_link` variants.



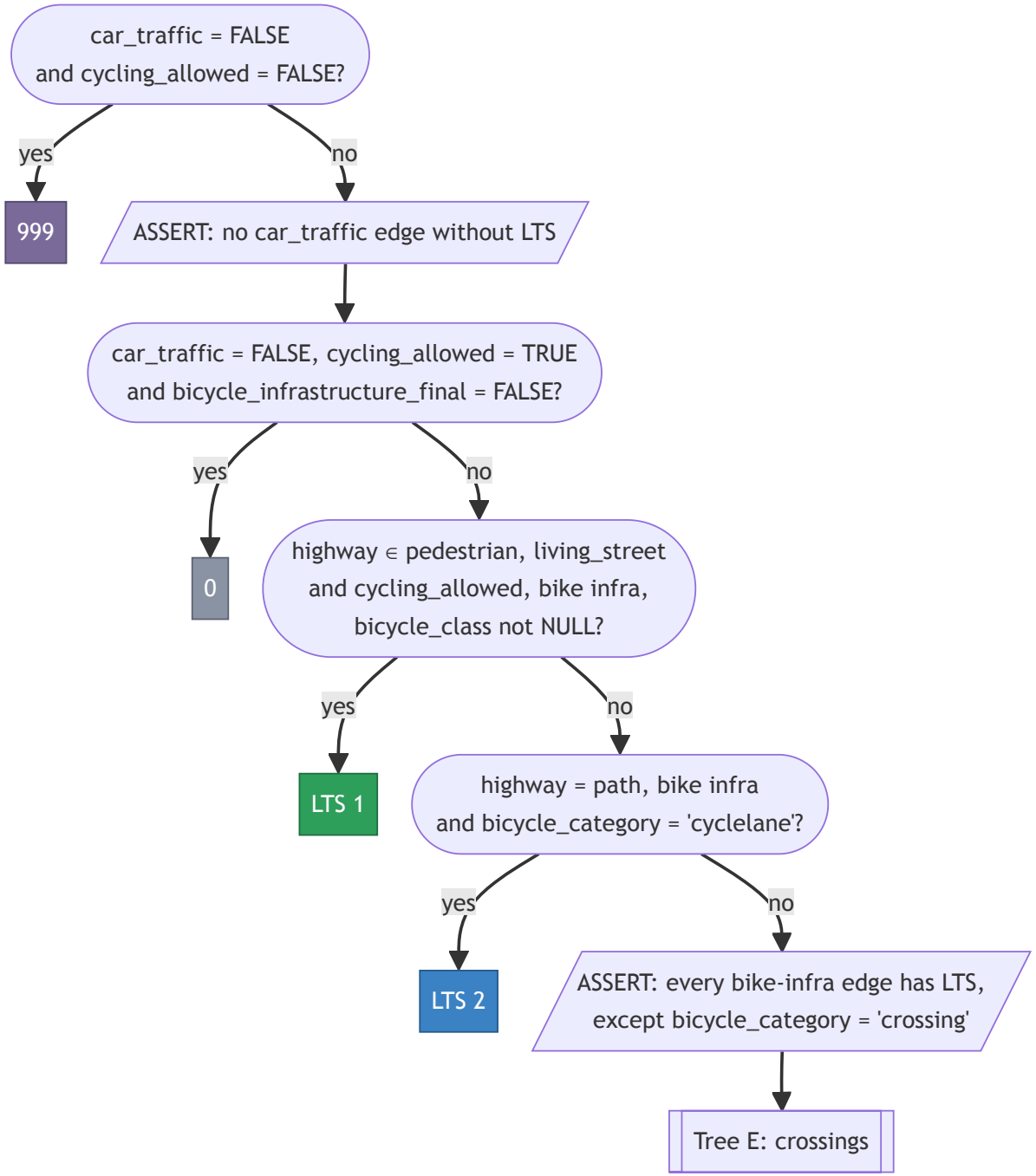
The “residential?” checks are the 10a override that turns LTS 3 into LTS 2 when `highway = 'residential'`, lanes < 4 and speed ≤ 50. With lanes = 4 the edge stays at LTS 3. When speed or lanes is NULL (only possible on non-car edges), a comparison with NULL is never true. Such

an edge only gets a score here if lanes > 4, if it is a motorway_link, or if it is a bus route with a known speed.

TREE D · END OF 10A

Fallbacks for edges with no score yet

These rules only touch edges where `lts IS NULL`, and they run in this order.


car_traffic = FALSE
and cycling_allowed = FALSE?
yes
no
999
ASSERT: no car_traffic edge without LTS
car_traffic = FALSE, cycling_allowed = TRUE
and bicycle_infrastructure_final = FALSE?
yes
no
0
highway ∈ pedestrian, living_street
and cycling_allowed, bike infra,
bicycle_class not NULL?
yes
no
LTS 1
highway = path, bike infra
and bicycle_category = 'cyclelane'?
yes
no
LTS 2
ASSERT: every bike-infra edge has LTS,
except bicycle_category = 'crossing'
Tree E: crossings

NODE LTS · 10B

Intersections

Each node in `nodes` takes the highest LTS among all edges whose `source` or `target` is that node. The order of precedence is:

4 › 3 › 2 › 1 › 0 › 999

999 has the lowest precedence. A node only gets 999 if every edge it touches is a 999 edge.

TREE E · 10C

Crossings

A crossing is an edge with `bicycle_infrastructure_final = TRUE`, `bicycle_category = 'crossing'` and `lts` still NULL. It takes the **highest LTS from 1 to 4** among all edges that touch its source or target node. In practice that is usually the road it crosses.

The node sets are built in 10b before any crossing gets a score, so crossings do not pass scores to each other. Nodes that only touch 0 or 999 edges are ignored. A crossing that touches no edge with LTS 1–4 stays NULL, and that fails the last assertion, *“Cycling edges missing LTS value”*.

Crossings whose speed and lanes already gave a score in Tree C (class 3) keep that score and skip this step.

EXPORT · 11C

From LTS to the `lts_viz` label

This step does not change `lts`. It builds the label that the exported map uses. The `CASE` takes the first row that matches. The `dirt_road` row is a separate `UPDATE` that runs afterwards and overwrites the label.

CONDITION	LTS_VIZ
1 and cycling_allowed	`all_cyclists`
2 and cycling_allowed	`most_cyclists`
3 and cycling_allowed	`confident_cyclists`
4 and cycling_allowed	`very_confident_cyclists`
1–4, cycling not allowed, `all_access`	`no_cycling`
999 and all_access	`pedestrian`
0 and all_access	`paths_bike`
`all_access = FALSE`	`no_access`
Overwrites the above: `highway = 'track'` and cycling_allowed	`dirt_road`

INPUTS

Where the tree's variables come from

The LTS rules read only a handful of columns. Each one is built in an earlier script.

MAXSPEED_ASSUMED · 09

OSM `maxspeed` starts with a number	that number
living_street	15
bicycle_road, cyclestreet, track (or tag =yes)	30
residential, service	50
trunk, primary, secondary, tertiary, unclassified (+_link)	80
motorway, motorway_link	130
No numeric maxspeed and `urban` ∈ urban, summerhouse, industrial (for residential/service/unclassified/trunk–tertiary)	50
Cleanup: 6→5, 12→15, 79→80	—

LANES_ASSUMED · 09

OSM `lanes`	as tagged
`lanes:forward` + `lanes:backward` (or whichever exists)	sum
residential, unclassified, service, tertiary, secondary, primary	2
trunk, motorway	6
cyclestreet, bicycle_road, living_street, track	2
path, bridleway	1
any `*_link`	2

COLUMN	SCRIPT	TRUE WHEN
`bus_route`	07	The OSM way belongs to a relation with `route = 'bus'`. Otherwise NULL, which the rules treat as false.
`bicycle_infrastructure_final`	04a	The edge is OSM bicycle infrastructure or was matched to GeoDanmark cycle infrastructure. Reset to FALSE for matched edges tagged `use_sidepath` / `separate`, and for motorway/motorroad edges that were only matched from GeoDanmark. 04b also sets it to TRUE for edges that fill gaps.
`car_traffic`	04a	`highway` is one of busway, trunk, primary, secondary, tertiary (+_link), living_street, residential, motorway (+_link), service, services, track, unclassified. Access must not be restricted, and motorcar/motor_vehicle must not be `no`.
`cycling_allowed`	04a	The bicycle tag is yes/permissive/ok/allowed/designated, *or* the edge is bike infrastructure, *or* it is a normal road/path type without bicycle = no/dismount/use_sidepath. Reset to FALSE for steps, restricted access, motorroad = yes, and roads with a separately mapped cycleway.

THINGS TO KNOW

Quirks in the current logic

1. **Some combinations have no rule.** Class 2 with speed 61–69. Class 2 with lanes ≥ 5 and speed < 50. Mixed traffic with speed < 30 and lanes = 4 and no bus route. Mixed traffic with speed 21–29 and lanes = 3 and no bus route. All of these fall through to the fallbacks. On car roads they would fail the assertion, so the pipeline relies on them never occurring.
2. **Some rules never change the result.** In the LTS 4 list, “mixed + bus + speed > 50” is already covered by “mixed + speed > 50”. In the LTS 3 list, the major-road rule says speed ≥ 50, but above 50 the edge becomes LTS 4 anyway, so it only matters at exactly 50 km/h. For class 2, the LTS 1 rule “speed ≤ 20 and lanes ≤ 3” is overwritten by LTS 2 when lanes = 3.

3. **Roads where cycling is banned still get LTS 1–4.** Examples are motorways and edges with a separate cycleway. They show up as `no_cycling` in `lts_viz`, but they count in the `lts` column and in node LTS. A motorway node therefore makes a nearby crossing LTS 4.
4. **The order of the statements matters.** LTS 1 → 2 → 3 → 4 run as separate overwrites. Reordering them in 10a, or adding an `AND lts IS NULL` guard, would change the results for many edges.
5. **Unclassified roads can only go from 4 down to 3.** The override requires `lts = 4`. In the same way, the residential override only lowers 3 to 2.

SOURCE

The raw rules as written in 10a

These are the conditions copied as written. In the code, a later list overwrites an earlier one.

▼ LTS 1 rules

```
bicycle_class = 1
OR (speed ≤ 30 AND lanes ≤ 2 AND (bike_infra OR highway NOT IN path,bridleway,footway,pedestrian,steps))
OR (speed ≤ 20 AND lanes ≤ 3 AND (same non-path condition))
OR (class = 2 AND speed ≤ 50 AND lanes ≤ 2)
```

▼ LTS 2 rules (overwrite 1)

```
(class = 2 AND speed ≤ 50 AND 3 ≤ lanes ≤ 4)
OR (class 3/NULL AND 30 < speed ≤ 50 AND lanes < 4)
OR (class 3/NULL AND 30 ≤ speed < 50 AND lanes = 3)
OR (class = 2 AND bus_route AND speed < 50)
OR (class 3/NULL AND bus_route AND speed ≤ 30)
```

▼ LTS 3 rules (overwrite 1–2)

```
(class = 2 AND 50 < speed ≤ 60 AND lanes < 5)
OR (class = 2 AND bus_route AND speed ≥ 50)
OR (class 3/NULL AND bus_route AND 30 < speed ≤ 50)
OR (class 3/NULL AND 30 ≤ speed ≤ 50 AND lanes = 4)
OR (class 3/NULL AND speed = 50 AND lanes = 3)
OR (class 3/NULL AND highway IN primary/secondary/tertiary(+_link)
    AND bike_infra = FALSE AND speed ≥ 50)
```

▼ LTS 4 rules (overwrite 1–3)

```
(class = 2 AND speed ≥ 50 AND lanes ≥ 5)
OR (class = 2 AND speed ≥ 70)
OR (class 3/NULL AND speed > 50)
OR (class 3/NULL AND lanes > 4)
OR (class 3/NULL AND bus_route AND speed > 50)
```

▼ Edge cases (overwrite everything above)

```
highway = motorway_link AND class NULL                          → 4
highway = unclassified AND class 3/NULL AND lanes < 3 AND lts = 4 → 3
highway = residential AND lanes < 4 AND speed ≤ 50 AND lts = 3  → 2
```